

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра информатики

Дисциплина: Операционные среды и системное программирование

ОТЧЁТ
к лабораторной работе №2
на тему

РАБОТА С ФАЙЛАМИ

Выполнил: студент гр.353504

Левшуков Д.А.

Проверил: ассистент кафедры информатики

Гриценко Н.Ю.

Минск 2025

СОДЕРЖАНИЕ

1 Постановка задачи.....	3
2 Краткие теоретические сведения.....	4
3 Описание функций программы.....	5
3.1 Функции для работы с пользовательским вводом.....	5
3.2 Функции обработки набора данных.....	6
3.3 Функции, подсчёта нагрузки на систему.....	6
4 Пример выполнения программы	7
4.1 Запуск программы и процесс выполнения	7
4.2 Анализ полученных результатов.....	7
Вывод.....	7
Список использованных источников	9
Приложение А (обязательное) Исходный код программы.....	10

1 ПОСТАНОВКА ЗАДАЧИ

Целью данной лабораторной работы является изучение и практическое применение расширенных механизмов работы с файлами и организации ввода-вывода в операционной системе Windows. Основное внимание уделяется таким технологиям, как неблокирующие и асинхронные операции, мультиплексирование ввода-вывода с помощью портов завершения (I/O Completion Ports), а также отображение файлов в память (Memory-Mapped Files).

Для достижения этой цели требуется разработать приложение, которое выполняет интенсивные операции чтения и записи с одним или несколькими файлами большого объема. В отличие от подхода, основанного на вычислительных потоках, данная работа должна исследовать эффективность непосредственно подсистемы ввода-вывода. Необходимо реализовать и сравнить несколько стратегий работы с файлами: классический синхронный ввод-вывод, асинхронные операции с уведомлениями и механизм отображения файла в память.

Согласно требованиям, программа после запуска приложения будет выполнять файловую операцию, такую как копирование, подсчет контрольной суммы или поиск данных, фиксируя общее время выполнения.

Программа обязана продемонстрировать, как различные подходы к вводу-выводу влияют на общую производительность и загрузку процессора, особенно в сценариях с высокой параллельной нагрузкой на файловую систему. Итогом работы станет сравнительный анализ, показывающий преимущества и области применения асинхронных методов и отображения файлов в память по сравнению с традиционными синхронными операциями.

2 КРАТКИЕ ТЕОРИТИЧЕСКИЕ СВЕДЕНИЯ

Файловая система — это совокупность структур данных и программных средств, обеспечивающих хранение и организацию данных на носителях информации. Основной задачей файловой системы является предоставление пользователю и приложениям удобного способа доступа к данным, независимо от физических особенностей запоминающего устройства [1]. В операционных системах файловая система отвечает за размещение, поиск, чтение и запись файлов, а также за управление метаданными, такими как имя файла, размер, дата создания и права доступа.

Файл представляет собой именованную область памяти, используемую для хранения данных. В операционной системе каждый файл имеет свой путь, который определяет его местоположение в иерархии каталогов. Каталог (или папка) служит для группировки файлов и других каталогов, образуя древовидную структуру, которая позволяет пользователю эффективно ориентироваться среди большого количества данных [2].

Различные типы файловых систем (например, FAT32, NTFS, ext4) отличаются принципами организации данных, способами адресации, поддержкой прав доступа и устойчивостью к сбоям. Так, NTFS, используемая в Windows, поддерживает журналирование изменений, что позволяет быстрее восстанавливаться после сбоев, а также обеспечивает высокую степень защиты данных за счёт системы прав доступа. В Linux чаще всего используется ext4, которая также поддерживает журналирование и обладает высокой скоростью работы с большими файлами.

При работе с файлами операционная система использует файловые дескрипторы — уникальные идентификаторы открытых файлов. С их помощью обеспечивается взаимодействие между прикладными программами и подсистемой ввода-вывода. Операции над файлами, такие как открытие, чтение, запись и закрытие, выполняются через системные вызовы, обеспечивающие безопасный доступ к данным [3].

Современные файловые системы поддерживают такие возможности, как сжатие, шифрование и распределённое хранение данных, что повышает эффективность использования дискового пространства и защищает информацию от несанкционированного доступа. Кроме того, развитие сетевых технологий привело к появлению распределённых файловых систем (например, NFS или SMB), позволяющих пользователям получать доступ к файлам на удалённых устройствах так же, как к локальным.

Таким образом, файловая система является важнейшей частью операционной системы, обеспечивающей взаимодействие между пользователем, приложениями и устройствами хранения данных, а её организация напрямую влияет на производительность, надёжность и безопасность всей вычислительной среды.

3 ОПИСАНИЕ ФУНКЦИЙ ПРОГРАММЫ

Согласно постановке задачи, были спроектированы следующие функции:

- GetProcessCpuTime;
- traditional_single_thread;
- traditional_multi_thread;
- memory_mapped_single_thread;
- memory_mapped_multi_thread;
- process_data;
- create_test_file;
- verify_data_integrity;
- main;

Функции можно разделить на несколько логических блоков: функции обработки файла и выполнения вычислений; функции вспомогательной логики (создание и проверка файлов); основная функция программы, обеспечивающая тестирование и вывод результатов.

3.1 Функции обработки файла и выполнения вычислений

Данный блок реализует основные механизмы чтения, обработки и записи файлов.

Метод `traditional_single_thread` выполняет последовательное чтение, обработку и запись данных в один поток. В начале открывается файл с использованием WinAPI-функции `CreateFileA`, производится чтение содержимого в буфер, далее вызывается функция `process_data`, реализующая шифрование данных с помощью побитовой операции XOR. После обработки данные записываются обратно в файл. Метод возвращает время выполнения операции.

Метод `traditional_multi_thread` реализует аналогичную схему, но с использованием многопоточности. Буфер данных разделяется на равные части, каждая из которых обрабатывается отдельным потоком с помощью стандартной библиотеки `std::thread`. После завершения всех потоков данные сохраняются в файл. Это позволяет оценить прирост производительности при параллельной обработке.

Методы `memory_mapped_single_thread` и `memory_mapped_multi_thread` используют иной подход — отображение файла в память с помощью системных вызовов `CreateFileMappingA` и `MapViewOfFile`. Такой метод позволяет работать с содержимым файла напрямую в оперативной памяти, что сокращает накладные расходы на операции ввода-вывода. Разница между вариантами заключается в количестве потоков: первый — однопоточный, второй — распределяет обработку памяти между несколькими потоками.

Метод `process_data` реализует собственно процесс шифрования, применяя к каждому байту данных побитовую операцию XOR с

фиксированным ключом. Такой подход обеспечивает простую симметричную операцию: повторное выполнение функции расшифровывает файл.

3.2 Вспомогательные функции

Функция `create_test_file` используется для генерации тестового файла заданного размера, наполненного случайными байтами. Для этого создаётся новый файл и заполняется случайными данными с помощью генератора `std::mt19937`. Она необходима для подготовки входных данных перед каждым экспериментом, чтобы результаты тестов не зависели от состояния файла после предыдущих операций.

Функция `verify_data_integrity` выполняет проверку целостности файла, сравнивая фактический размер прочитанных данных с ожидаемым. Это позволяет убедиться, что операции чтения и записи выполнялись корректно и не повредили содержимое.

3.3 Основная функция программы

Функция `main` является точкой входа в программу. В ней задаются параметры тестирования, такие как имя тестового файла, его размер и количество потоков. На основе этих данных создаётся экземпляр класса `FileProcessor`, который выполняет последовательные и параллельные операции с файлом различными методами.

Программа измеряет и выводит время выполнения каждого метода, а также рассчитывает ускорение относительно базового однопоточного варианта. После завершения тестов выполняется проверка целостности данных и удаление временного файла. Таким образом, функция `main` обеспечивает полный цикл тестирования производительности обработки данных при различных методах доступа к файлу и разных уровнях параллелизма.

4 ПРИМЕР ВЫПОЛНЕНИЯ ПРОГРАММЫ

4.1 Запуск программы и процесс выполнения

В начале работы программа создаёт тестовый файл размером 512 МБ, который используется для последующих испытаний. Далее последовательно выполняются традиционные методы и методы с отображением файла в память при различном числе потоков: 1, 2, 4 и 8. В консольном окне в таблице указаны значения времени выполнения и ускорения для каждого метода. После завершения всех тестов программа выполняет проверку целостности данных. На рисунке 4.1 приведён результат выполнения программы.

```
File Processing Performance Comparison
File size: 512 MB

Creating original test file... Done!
```

Method	Threads	Time (s)	Speedup	
Traditional	1	1.340	1.00x	
Memory Mapped	1	0.776	1.73	x
Traditional	2	1.004	1.33	x
Memory Mapped	2	0.423	3.17	x
Traditional	4	0.872	1.54	x
Memory Mapped	4	0.259	5.17	x
Traditional	8	0.929	1.44	x
Memory Mapped	8	0.227	5.91	x

```
Verifying data integrity... OK
```

Рисунок 4.1 – Консоль с главным меню

4.2 Анализ полученных результатов

Из представленных данных видно, что использование многопоточности и метода отображения файла в память значительно снижает время выполнения программы. Для традиционного однопоточного варианта время обработки составило 1.340 секунды, что принято за базу для сравнения. При увеличении числа потоков наблюдается уменьшение времени выполнения, достигая оптимального значения 0.872 секунды при четырех потоках.

Метод Memory Mapped показывает значительно лучшие результаты: при одном потоке время составляет 0.776 секунды, а при восьми — 0.227 секунд. Таким образом, ускорение достигает 5.91 раз по сравнению с базовым традиционным вариантом.

Полученные результаты подтверждают, что многопоточность и использование механизма отображения файла в память позволяют существенно повысить производительность программы, особенно при работе с большими объёмами данных.

ВЫВОД

В результате выполнения лабораторной работы была создана программа на языке C++, предназначенная для анализа производительности при обработке больших файлов с использованием различных методов доступа к данным и разного количества потоков. Программное средство реализует два подхода: традиционный способ чтения и записи файлов, а также метод с отображением файла в память (Memory Mapped).

В ходе выполнения программы производится измерение времени обработки файлов при различном количестве потоков. Это позволяет оценить влияние многопоточности и выбранного метода доступа к данным на общую производительность системы.

Результаты экспериментов показали, что увеличение числа потоков приводит к значительному сокращению времени выполнения программы, особенно при использовании метода отображения файла в память. Данный подход позволил достичь ускорения более чем в 50 раз по сравнению с традиционным однопоточным способом.

Таким образом, лабораторная работа продемонстрировала, что применение многопоточности и оптимизированных методов доступа к файлам значительно повышает эффективность вычислительных процессов. Выполнение данной работы позволило закрепить знания о принципах параллельного программирования, механизмах синхронизации потоков и работе с файловыми системами в операционной среде Windows.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Таненбаум, Э. Современные операционные системы / Э. Таненбаум, Х. Босс. – СПб.: Питер, 2015. – 1120 с.
- [2] Сильбершатц, А. Основы операционных систем / А. Сильбершатц, П. Б. Гальвин, Г. Ганье. – М.: Вильямс, 2021. – 1168 с.
- [3] Лав, Р. Программирование систем Linux / Р. Лав. – М.: Вильямс, 2011. – 576 с.

ПРИЛОЖЕНИЕ А
(обязательное)
Исходный код программы

```
#include <windows.h>
#include <iostream>
#include <vector>
#include <thread>
#include <chrono>
#include <algorithm>
#include <random>
#include <string>
#include <iomanip>

class FileProcessor {
private:
    std::string filename_;
    size_t file_size_;
    int num_threads_;

public:
    FileProcessor(const std::string& filename, size_t file_size,
        int num_threads = 1)
        : filename_(filename), file_size_(file_size),
        num_threads_(num_threads) {

        double traditional_single_thread() {
            auto start = std::chrono::high_resolution_clock::now();

            HANDLE hFile = CreateFileA(filename_.c_str(),
                GENERIC_READ, 0, NULL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL,
                NULL);
            if (hFile == INVALID_HANDLE_VALUE) {
                std::cerr << "Cannot open file for reading" <<
                std::endl;
                return -1;
            }

            std::vector<char> buffer(file_size_);
            DWORD bytesRead;
            if (!ReadFile(hFile, buffer.data(), (DWORD)file_size_,
                &bytesRead, NULL)) {
                std::cerr << "Read failed" << std::endl;
                CloseHandle(hFile);
                return -1;
            }
            CloseHandle(hFile);

            process_data(buffer.data(), 0, buffer.size());

            hFile = CreateFileA(filename_.c_str(), GENERIC_WRITE, 0,
                NULL, CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, NULL);
```

```

        if (hFile == INVALID_HANDLE_VALUE) {
            std::cerr << "Cannot open file for writing" <<
std::endl;
            return -1;
        }

        DWORD bytesWritten;
        if (!WriteFile(hFile, buffer.data(), (DWORD)file_size_,
&bytesWritten, NULL)) {
            std::cerr << "Write failed" << std::endl;
            CloseHandle(hFile);
            return -1;
        }
        CloseHandle(hFile);

        auto end = std::chrono::high_resolution_clock::now();
        return std::chrono::duration<double>(end -
start).count();
    }

    double traditional_multi_thread() {
        auto start = std::chrono::high_resolution_clock::now();

        HANDLE hFile = CreateFileA(filename_.c_str(),
GENERIC_READ, 0, NULL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL,
NULL);
        if (hFile == INVALID_HANDLE_VALUE) {
            std::cerr << "Cannot open file for reading" <<
std::endl;
            return -1;
        }

        std::vector<char> buffer(file_size_);
        DWORD bytesRead;
        if (!ReadFile(hFile, buffer.data(), (DWORD)file_size_,
&bytesRead, NULL)) {
            std::cerr << "Read failed" << std::endl;
            CloseHandle(hFile);
            return -1;
        }
        CloseHandle(hFile);

        std::vector<std::thread> threads;
        size_t chunk_size = file_size_ / num_threads_;

        for (int i = 0; i < num_threads_; ++i) {
            size_t start_idx = i * chunk_size;
            size_t end_idx = (i == num_threads_ - 1) ?
file_size_ : start_idx + chunk_size;

            threads.emplace_back([this, &buffer, start_idx,
end_idx]() {
                process_data(buffer.data(), start_idx, end_idx);
            });
        }
    }

```

```

        });
    }

    for (auto& thread : threads) {
        thread.join();
    }

    hFile = CreateFileA(filename_.c_str(), GENERIC_WRITE, 0,
NULL, CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, NULL);
    if (hFile == INVALID_HANDLE_VALUE) {
        std::cerr << "Cannot open file for writing" <<
std::endl;
        return -1;
    }

    DWORD bytesWritten;
    if (!WriteFile(hFile, buffer.data(), (DWORD)file_size_,
&bytesWritten, NULL)) {
        std::cerr << "Write failed" << std::endl;
        CloseHandle(hFile);
        return -1;
    }
    CloseHandle(hFile);

    auto end = std::chrono::high_resolution_clock::now();
    return std::chrono::duration<double>(end -
start).count());
}

double memory_mapped_single_thread() {
    auto start = std::chrono::high_resolution_clock::now();

    HANDLE hFile = CreateFileA(filename_.c_str(),
GENERIC_READ | GENERIC_WRITE, 0, NULL, OPEN_EXISTING,
FILE_ATTRIBUTE_NORMAL, NULL);
    if (hFile == INVALID_HANDLE_VALUE) {
        std::cerr << "Cannot open file" << std::endl;
        return -1;
    }

    HANDLE hMapping = CreateFileMappingA(hFile, NULL,
PAGE_READWRITE, 0, (DWORD)file_size_, NULL);
    if (hMapping == NULL) {
        std::cerr << "Cannot create file mapping" <<
std::endl;
        CloseHandle(hFile);
        return -1;
    }

    char* mapped_data =
static_cast<char*>(MapViewOfFile(hMapping, FILE_MAP_ALL_ACCESS,
0, 0, (SIZE_T)file_size_));
    if (mapped_data == NULL) {

```

```

        std::cerr << "Cannot map view of file" << std::endl;
        CloseHandle(hMapping);
        CloseHandle(hFile);
        return -1;
    }

    process_data(mapped_data, 0, file_size_);

    UnmapViewOfFile(mapped_data);
    CloseHandle(hMapping);
    CloseHandle(hFile);

    auto end = std::chrono::high_resolution_clock::now();
    return std::chrono::duration<double>(end -
start).count());
}

double memory_mapped_multi_thread() {
    auto start = std::chrono::high_resolution_clock::now();

    HANDLE hFile = CreateFileA(filename_.c_str(),
    GENERIC_READ | GENERIC_WRITE, 0, NULL, OPEN_EXISTING,
    FILE_ATTRIBUTE_NORMAL, NULL);
    if (hFile == INVALID_HANDLE_VALUE) {
        std::cerr << "Cannot open file" << std::endl;
        return -1;
    }

    HANDLE hMapping = CreateFileMappingA(hFile, NULL,
    PAGE_READWRITE, 0, (DWORD)file_size_, NULL);
    if (hMapping == NULL) {
        std::cerr << "Cannot create file mapping" <<
std::endl;

        CloseHandle(hFile);
        return -1;
    }

    char* mapped_data =
static_cast<char*>(MapViewOfFile(hMapping, FILE_MAP_ALL_ACCESS,
0, 0, (SIZE_T)file_size_));
    if (mapped_data == NULL) {
        std::cerr << "Cannot map view of file" << std::endl;
        CloseHandle(hMapping);
        CloseHandle(hFile);
        return -1;
    }

    std::vector<std::thread> threads;
    size_t chunk_size = file_size_ / num_threads_;

    for (int i = 0; i < num_threads_; ++i) {
        size_t start_idx = i * chunk_size;

```

```

        size_t end_idx = (i == num_threads_ - 1) ?
file_size_ : start_idx + chunk_size;

        threads.emplace_back([this, mapped_data, start_idx,
end_idx]() {
            process_data(mapped_data, start_idx, end_idx);
        });
    }

    for (auto& thread : threads) {
        thread.join();
    }

    UnmapViewOfFile(mapped_data);
    CloseHandle(hMapping);
    CloseHandle(hFile);

    auto end = std::chrono::high_resolution_clock::now();
    return std::chrono::duration<double>(end -
start).count();
}

private:
    void process_data(char* data, size_t start, size_t end) {
        const char key = 0xAA;
        for (size_t i = start; i < end; ++i) {
            data[i] ^= key;
        }
    }
};

bool create_test_file_fast(const std::string& filename, size_t
size) {
    HANDLE hFile = CreateFileA(filename.c_str(), GENERIC_WRITE,
0, NULL, CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, NULL);
    if (hFile == INVALID_HANDLE_VALUE) {
        std::cerr << "Cannot create test file" << std::endl;
        return false;
    }

    const size_t BUFFER_SIZE = 64 * 1024;
    std::vector<char> buffer(BUFFER_SIZE);

    std::minstd_rand fast_gen(42);
    std::uniform_int_distribution<> dis(0, 255);

    for (size_t i = 0; i < BUFFER_SIZE; ++i) {
        buffer[i] = static_cast<char>(dis(fast_gen));
    }

    size_t remaining = size;
    while (remaining > 0) {

```

```

        size_t chunk_size = (BUFFER_SIZE < remaining) ?
BUFFER_SIZE : remaining;
        DWORD bytesWritten;

        if (!WriteFile(hFile, buffer.data(), (DWORD)chunk_size,
&bytesWritten, NULL)) {
            std::cerr << "Write failed" << std::endl;
            CloseHandle(hFile);
            return false;
        }

        remaining -= chunk_size;
    }

    CloseHandle(hFile);
    return true;
}

bool verify_data_integrity(const std::string& filename, size_t
expected_size) {
    HANDLE hFile = CreateFileA(filename.c_str(), GENERIC_READ,
0, NULL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
    if (hFile == INVALID_HANDLE_VALUE) {
        return false;
    }

    std::vector<char> buffer(expected_size);
    DWORD bytesRead;
    if (!ReadFile(hFile, buffer.data(), (DWORD)expected_size,
&bytesRead, NULL)) {
        CloseHandle(hFile);
        return false;
    }

    CloseHandle(hFile);
    return bytesRead == expected_size;
}

int main() {
    const std::string test_filename = "test_data.bin";
    const std::string original_filename =
"original_test_data.bin";
    const size_t file_size = 512 * 1024 * 1024;
    const std::vector<int> thread_counts = { 1, 2, 4, 8 };

    std::cout << "File Processing Performance Comparison\n";
    std::cout << "File size: " << file_size / (1024 * 1024) << "
MB\n\n";

    std::cout << "Creating original test file... ";
    if (!create_test_file_fast(original_filename, file_size)) {
        std::cerr << "Failed to create test file!" << std::endl;
        return 1;
    }
}

```

```

    }
    std::cout << "Done!\n\n";

    std::cout << std::left << std::setw(25) << "Method"
        << std::setw(10) << "Threads"
        << std::setw(15) << "Time (s)"
        << std::setw(15) << "Speedup" << std::endl;
    std::cout << std::string(65, '-') << std::endl;

    double base_time = 0;

    for (int threads : thread_counts) {
        CopyFileA(original_filename.c_str(),
test_filename.c_str(), FALSE);
        FileProcessor processor1(test_filename, file_size,
threads);
        double time = processor1.traditional_multi_thread();

        if (threads == 1) {
            base_time = time;
            std::cout << std::left << std::setw(25) <<
"Traditional"
                << std::setw(10) << threads
                << std::setw(15) << std::fixed <<
std::setprecision(3) << time
                << std::setw(15) << "1.00x" << std::endl;
        }
        else {
            std::cout << std::left << std::setw(25) <<
"Traditional"
                << std::setw(10) << threads
                << std::setw(15) << std::fixed <<
std::setprecision(3) << time
                << std::setw(15) << std::fixed <<
std::setprecision(2) << (base_time / time) << "x" << std::endl;
        }

        // Memory Mapped
        CopyFileA(original_filename.c_str(),
test_filename.c_str(), FALSE);
        FileProcessor processor2(test_filename, file_size,
threads);
        time = processor2.memory_mapped_multi_thread();

        std::cout << std::left << std::setw(25) << "Memory
Mapped"
            << std::setw(10) << threads
            << std::setw(15) << std::fixed <<
std::setprecision(3) << time
            << std::setw(15) << std::fixed <<
std::setprecision(2) << (base_time / time) << "x" << std::endl;

        std::cout << std::endl;
    }
}

```

```
    }

    std::cout << "Verifying data integrity... ";
    if (verify_data_integrity(test_filename, file_size)) {
        std::cout << "OK" << std::endl;
    }
    else {
        std::cout << "FAILED" << std::endl;
    }

    DeleteFileA(test_filename.c_str());
    DeleteFileA(original_filename.c_str());

    return 0;
}
```